

FundsRoom Operations Portal

Complete project documentation

Wholesale operations workspace

Sales, warehouse, accounts, administration, inventory, challans, notifications, customer relationships, and stock-request approvals in one role-aware system.

Live links

[Live frontend on Vercel](#)

[GitHub repository](#)

This document is generated from the repository README and includes full-screen product captures.

FundsRoom is a full-stack mini ERP and CRM operations portal for a wholesale business. It connects customer relationships, sales challans, inventory, warehouse movements, notifications, and sales reporting in one workspace.

The application is intentionally built as a practical case-study project: the workflows are real, the data is stored in MySQL, the API enforces roles and validation, and the interface is designed around the daily work of sales, warehouse, accounts, and administration teams.

What the application does

The portal covers the complete path from a customer relationship to a confirmed sale:

1. A team member signs in with a role-based account.
2. Sales can search or create a customer record.
3. A sales user creates a draft challan for one customer and one or more products.
4. The API stores a product and price snapshot on every challan item.
5. Confirming the challan checks inventory inside a database transaction.
6. The API refuses confirmation when stock would become negative.
7. Successful confirmation decreases stock and records an **OUT** movement.

8. The dashboard refreshes sales totals, low-stock warnings, follow-ups, notifications, and charts.
9. Accounts or sales can download a PDF challan and export sales data as CSV.

The customer workspace also keeps follow-up notes, customer queries, query status, priorities, and reviews together so a relationship is not reduced to a single row in a table.

See SECURITY.md for the role-permission matrix, threat model, implemented controls, and production hardening checklist.

Deployment

- [Live frontend on Vercel](#)
- [Backend deployment on Vercel](#)
- [GitHub repository](#)

The frontend is deployed and available at the link above. The backend deployment is ready for production environment variables, including a managed MySQL connection. A local MySQL server at `localhost` cannot be reached by Vercel serverless functions; set the backend's `DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER`, `DB_PASSWORD`, `JWT_SECRET`, and `CLIENT_ORIGIN` values in Vercel before enabling live sign-in.

Screenshots

![[Sales overview]](docs/screenshots/sales-overview.png)

![[Sales products and inventory]](docs/screenshots/sales-products.png)

![[Warehouse overview]](docs/screenshots/warehouse-overview.png)

For a print-ready version with full-page screenshots and clickable links, see [README-FundsRoom.pdf](output/pdf/README-FundsRoom.pdf).

Feature inventory

Authentication and roles

- JWT-based login with a 12-hour token.
- Password hashing with `bcryptjs`.
- Four roles: `ADMIN`, `SALES`, `WAREHOUSE`, and `ACCOUNTS`.
- Route-level authorization for actions that change business data.
- Consistent `401`, `403`, `404`, `409`, and `422` API responses.

Dashboard

- Customer, product, and low-stock metrics.
- Low-stock inventory watch list.
- Upcoming customer follow-ups.
- Recent challans.
- Daily sales chart for the selected reporting window.
- Sales-by-teammate chart.
- Sales chart hover details: seller, customer, business, product, SKU, quantity, and line value.
- Persistent notification inbox with unread count.
- Notification types for low stock, follow-ups, and open customer queries.
- Automatic refresh of workspace data every 30 seconds.

Customer relationship management

- Customer fields: name, business, mobile, email, GST number, customer type, address, status, and follow-up date.
- Customer types: retail, wholesale, and distributor.
- Customer statuses: lead, active, and inactive.
- Fast debounced search across customer name, business name, and mobile number.
- Customer detail view with three tabs:
 - Timeline and follow-up notes
 - Queries with priority and resolve action
 - Reviews with one-to-five-star ratings

Product and inventory management

- Product fields: name, SKU, category, unit price, current stock, minimum alert quantity, and warehouse location.
- Search across product name, SKU, and category.
- Low-stock status derived from `current_stock <= minimum_stock_alert_quantity`.
- Manual stock receipt workflow with a reason.
- Stock movement history with type, quantity, reason, reference, user, and timestamp.
- Transaction-safe stock adjustments.
- Negative stock is rejected with a conflict response.

Sales challans

- Draft, confirmed, and cancelled states.
- Automatic challan numbering for records created through the API.
- Multiple products in a single challan.
- Duplicate products in one challan are rejected during validation.

- Draft save and draft cancellation.
- Confirmation locks the challan and product rows before checking stock.
- Confirmation stores product name, SKU, unit price, and quantity snapshots.
- Confirmation creates linked stock **OUT** movements.
- Challan search across number, customer, and business.
- Challan detail view with item lines and actions.
- PDF document download for each challan.
- CSV export for filtered sales data.

Notifications

Notifications are persisted in MySQL rather than being hardcoded in the React interface.

- Low-stock notifications are generated from current product stock.
- Follow-up notifications are generated for follow-ups due today through the next seven days.
- Open customer queries become notifications with their priority.
- Notifications keep a **read_at** timestamp.
- Individual notification read action.
- Mark-all-as-read action.
- Resolved queries and recovered stock warnings are removed during reconciliation.
- New issues become unread again when their underlying condition returns.

Architecture

```

React + Vite frontend
  |
  | JSON over REST / Bearer JWT
  v
Express + TypeScript API
  |
  | mysql2 connection pool
  v
Local MySQL database
  |
  |-- users and roles
  |-- customers and CRM engagement
  |-- products and stock movements
  |-- challans and item snapshots
  |-- persisted notifications

```

Repository structure

```

fundsroom/
├── backend/
│   ├── database/
│   │   ├── schema.sql
│   │   ├── crm-engagement.sql
│   │   ├── notifications.sql
│   │   └── src/
│   ├── config/
│   └── database/
└── # Environment parsing
    # MySQL pool and transactions

```

■ ■ ■■ middleware/	# Auth, roles, and errors
■ ■ ■■ routes/	# REST API modules
■ ■ ■■ scripts/	# Database setup and demo data
■ ■ ■■ server.ts	
■ ■■ .env.example	
■ ■■ package.json	
■■■ frontend/	
■ ■■ src/	
■ ■ ■■ App.tsx	# Login, shell, dashboard, challan modal
■ ■ ■■ Workspace.tsx	# CRM, products, stock, challan screens
■ ■ ■■ lib/api.ts	# Authenticated API helpers and downloads
■ ■ ■■ styles.css	# Responsive visual system
■ ■■ index.html	
■ ■■ package.json	
■■■ README.md	

Prerequisites

- Node.js 20 or newer
- MySQL 8 or newer
- npm
- A MySQL user that can create tables and a database during setup

Docker is not required. The project uses a local MySQL server through MySQL Workbench or the MySQL service installed on Windows.

Installation

Open two PowerShell windows from the repository root.

1. Install backend dependencies

```
cd backend
npm.cmd install
```

2. Configure the backend

Copy the example environment file:

```
Copy-Item .env.example .env
```

Set the values in **backend/.env**:

```
PORT=4000
DB_HOST=localhost
DB_PORT=3306
DB_NAME=fundsroom
DB_USER=root
DB_PASSWORD=your-local-mysql-password
JWT_SECRET=replace-with-a-long-random-secret
CLIENT_ORIGIN=http://localhost:5173
```

The real **.env** file is ignored by Git. Never commit a database password or JWT secret.

3. Create and prepare MySQL

From `backend/`:

```
npm.cmd run db:create
npm.cmd run db:setup
npm.cmd run db:crm
npm.cmd run db:notifications
npm.cmd run db:crm-replies
npm.cmd run db:stock-requests
npm.cmd run db:seed
npm.cmd run db:demo-history
```

What each command does:

Command	Purpose
<code>db:create</code>	Creates the configured MySQL database if it does not exist.
<code>db:setup</code>	Creates the core users, customers, products, stock, challan, and item tables.
<code>db:crm</code>	Adds customer queries and reviews.
<code>db:notifications</code>	Adds the persisted notification inbox table.
<code>db:crm-replies</code>	Adds the customer-query reply thread table.
<code>db:stock-requests</code>	Adds Sales-to-Warehouse stock requests and approval tracking.
<code>db:seed</code>	Adds the four demo accounts and starter catalogue data.
<code>db:demo-history</code>	Adds ten days of repeat-safe sales, stock movements, queries, and reviews.

The `demo-history` script is additive and checks for its own `DEM010` - records before inserting them again.

4. Install frontend dependencies

In the second PowerShell window:

```
cd frontend
npm.cmd install
```

Running the application

Backend:

```
cd backend
npm.cmd run dev
```

Frontend:

```
cd frontend
npm.cmd run dev
```

Open <http://localhost:5173>.

The API health check is available at <http://localhost:4000/health>.

Demo accounts

All seeded demo accounts use the password **FundsRoom@123**.

| Email | Role | Main responsibilities |

| --- | --- | --- |

| **admin@stockflow.test** | Admin | Operational oversight, query resolution, approvals, and full read access. |

| **sales@stockflow.test** | Sales | Customers, incoming queries, customer replies/feedback, drafts, and challan confirmation. |

| **warehouse@stockflow.test** | Warehouse | Products, stock receipts or adjustments, Sales stock-request approvals, challans, and movement notes. |

| **accounts@stockflow.test** | Accounts | Sales review, challan viewing, PDF, and CSV export. |

These credentials are for local demonstration only.

REST API reference

Every endpoint below, except login and health, requires:

```
Authorization: Bearer <jwt-token>
```

Authentication

```
POST /api/auth/login
Content-Type: application/json

{
  "email": "sales@stockflow.test",
  "password": "FundsRoom@123"
}
```

The response contains a signed JWT and the authenticated user payload:

```
{
  "token": "eyJ...",
  "user": {
    "id": "uuid",
    "name": "Riya Kapoor",
    "email": "sales@stockflow.test",
    "role": "SALES"
  }
}
```

Dashboard and analytics

```
GET /api/dashboard
GET /api/analytics/sales?days=30
GET /api/notifications
```

The analytics response contains three complementary views:

```
{
  "data": {
    "days": 30,
    "daily": [
      { "saleDate": "2026-08-08", "challans": 2, "revenue": "950.00", "units": "4" }
    ]
  }
}
```

```

    ],
    "byUser": [
      { "userName": "Riya Kapoor", "challans": 3, "revenue": "1425.00", "units": "8" }
    ],
    "details": [
      {
        "challanNumber": "DEM010-20260808-01",
        "sellerName": "Riya Kapoor",
        "customerName": "Priya Sharma",
        "businessName": "Evergreen Stores",
        "productName": "A4 Copier Paper - 75 GSM",
        "quantity": 2,
        "revenue": "570.00"
      }
    ]
  }
}
}

```

The `details` array is what powers the chart hover explanation. It prevents a chart from hiding the transactions behind a single number.

Customers and CRM

```

GET    /api/customers?search=Evergreen&limit=20
POST   /api/customers
GET     /api/customers/:customerId
PUT     /api/customers/:customerId
POST    /api/customers/:customerId/follow-ups
GET     /api/customers/:customerId/queries
POST    /api/customers/:customerId/queries
PATCH  /api/customers/:customerId/queries/:queryId/resolve
GET     /api/customers/:customerId/reviews
POST    /api/customers/:customerId/reviews

```

Example customer creation:

```

{
  "customerName": "Aditi Singh",
  "businessName": "Central Stationers",
  "mobile": "9876500000",
  "email": "aditi@example.com",
  "customerType": "WHOLESALE",
  "status": "LEAD",
  "address": "21 Market Street, Chandigarh",
  "gstNumber": null,
  "followUpDate": "2026-08-14"
}

```

Example query:

```

{
  "subject": "Delivery timing",
  "message": "Can the order arrive before Friday?",
  "priority": "HIGH"
}

```

Products and stock

```

GET    /api/products?search=paper&limit=20
POST   /api/products
PUT     /api/products/:productId
POST    /api/products/:productId/stock-movements
GET     /api/products/:productId/stock-movements

```

Manual receipt example:


```
{
  "quantity": 25,
  "movementType": "IN",
  "reason": "Supplier delivery received"
}
```

An OUT request is rejected when its quantity exceeds the locked current stock. The stock check and update happen in one transaction so two users cannot silently oversell the same units.

Challans, PDF, and export

```
GET    /api/challans?search=Priya&status=CONFIRMED
POST   /api/challans
GET    /api/challans/:challanId
PATCH /api/challans/:challanId/confirm
PATCH /api/challans/:challanId/cancel
GET    /api/challans/:challanId/pdf
GET    /api/challans/export.csv
```

Create-draft example:

```
{
  "customerId": "customer-uuid",
  "items": [
    { "productId": "product-uuid", "quantity": 3 },
    { "productId": "another-product-uuid", "quantity": 1 }
  ]
}
```

Confirmation performs these steps in order:

```
lock challan
-> load and lock product rows
-> compare every requested quantity with current stock
-> reject the whole transaction if any item is short
-> decrement each product
-> write OUT stock movements
-> set challan status to CONFIRMED
```

The PDF endpoint streams a generated sales challan document. The CSV endpoint includes challan number, customer, business, status, units, creation time, and creator.

Notifications

```
GET    /api/notifications
PATCH /api/notifications/:notificationId/read
PATCH /api/notifications/read-all
```

The response includes both the records and the current unread count:

```
{
  "data": [
    {
      "id": "uuid",
      "type": "LOW_STOCK",
      "title": "Low stock:",
      "detail": "Permanent Marker - Black has 6 units left",
      "priority": "HIGH",
      "readAt": null,
      "createdAt": "2026-08-09T08:00:00.000Z"
    }
  ],
  "unreadCount": 1
}
```

Data model

The MySQL schema is split into business areas:

Table	Responsibility
`users`	Identity, password hash, and role.
`customers`	Customer and business profile.
`customer_follow_ups`	Timeline notes and follow-up dates.
`customer_queries`	Support or sales questions, priorities, and resolution state.
`customer_reviews`	One-to-five-star relationship feedback.
`products`	Catalogue, price, warehouse, current stock, and alert threshold.
`stock_movements`	Immutable IN/OUT audit entries.
`challans`	Sales document header and lifecycle status.
`challan_items`	Product and price snapshots belonging to a challan.
`challan_sequence`	Auto-number sequence for API-created challans.
`notifications`	Reconciled alerts and read timestamps.

Foreign keys protect customer, product, user, and challan relationships. Check constraints protect positive quantities, non-negative stock, and valid review ratings.

Validation and error handling

Zod schemas validate request bodies before they reach the database. Examples include:

- A customer name must be between two and 160 characters.
- A product SKU must be present and unique.
- A price and opening stock cannot be negative.
- A challan must contain at least one item.
- A product cannot appear twice in the same challan.
- A query priority must be `LOW`, `MEDIUM`, or `HIGH`.
- A review rating must be an integer from one to five.

The API translates errors into predictable responses:

```
{
  "message": "Insufficient stock for Blue Ball Pen - Pack of 10. Available: 8, requested: 100."
}
```

Verification commands

Run these before sharing a change:

```
cd backend
npm.cmd run typecheck

cd ..\frontend
npm.cmd run build
```

For a live smoke test:

```
Invoke-RestMethod http://localhost:4000/health
```

Recommended manual checks:

1. Log in as Sales.
2. Search for a customer and open its profile.
3. Add a query and resolve it.
4. Open the notification bell and mark one item read.
5. Create a draft challan and confirm it.
6. Confirm that stock decreased and an OUT movement exists.
7. Search for the challan.
8. Download its PDF and export the CSV.
9. Hover a daily sales bar to inspect the underlying seller, customer, product, and quantity.
10. Log in as Warehouse and record an IN stock movement.

Deployment notes

The local setup uses MySQL on `localhost`, but the application is not tied to local storage. For deployment:

1. Create a managed MySQL 8 database.
2. Set `DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER`, and `DB_PASSWORD` to the provider values.
3. Set a strong production `JWT_SECRET`.
4. Set `CLIENT_ORIGIN` to the deployed frontend URL.
5. Run the database setup and migration scripts once against the managed database.
6. Build the frontend with `npm run build`.
7. Start the API with `npm run start` and serve the frontend `dist` directory through a static host.

The backend uses a connection pool and environment variables, so the database can move from a local MySQL service to a managed MySQL service without changing the application code.

Current boundaries and next production steps

This is a focused case-study implementation rather than a complete accounting suite. Before production use, the next steps would be:

- Add refresh-token rotation and server-side session revocation.

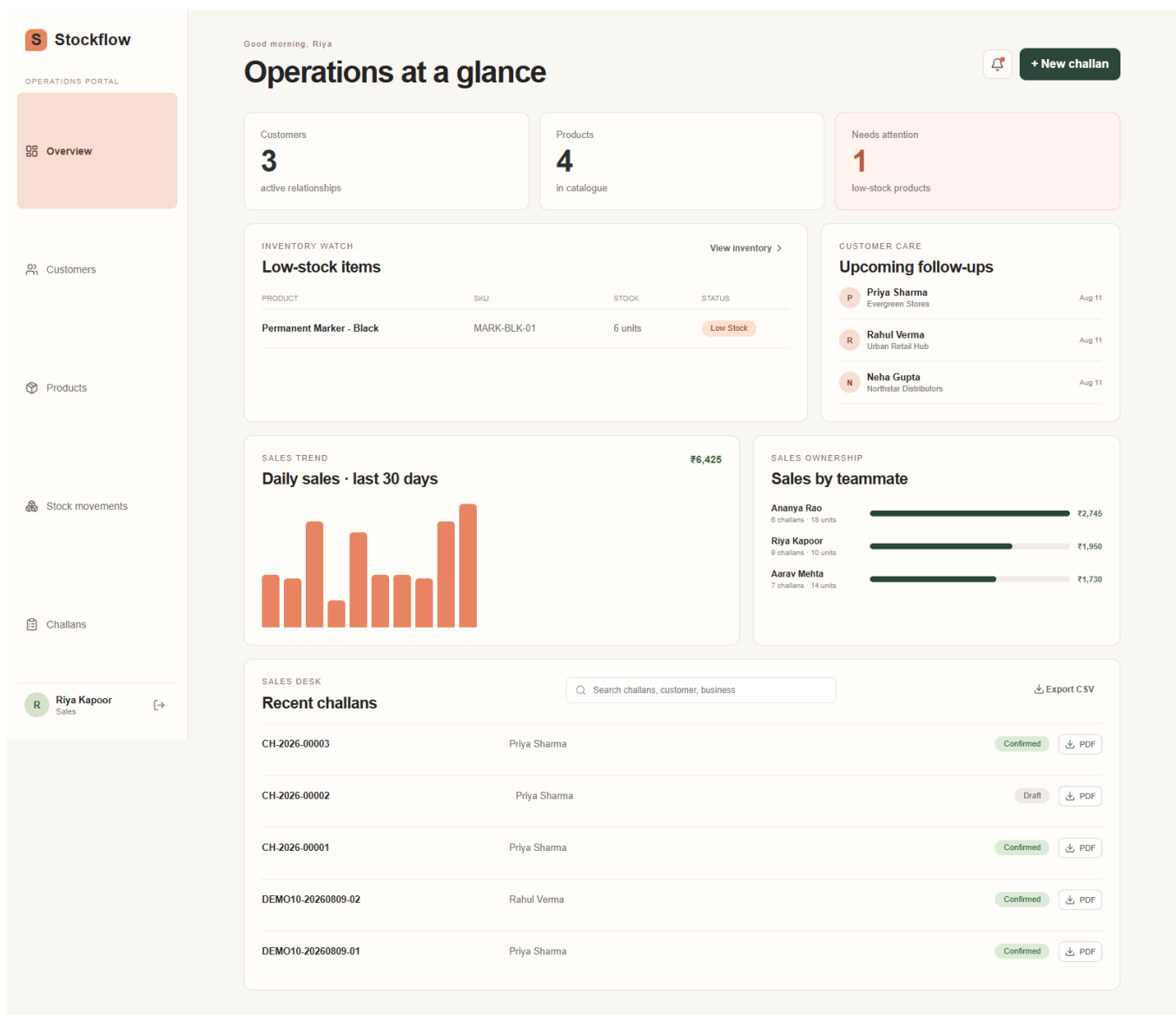
- Add audit events for edits and permission changes.
- Add pagination controls to every large table.
- Add automated API integration tests and browser end-to-end tests to CI.
- Add configurable tax, discount, shipping, and payment fields to invoices.
- Add object storage for product images and PDF archival.
- Add a managed queue or WebSocket channel for instant notification delivery.
- Add database backups, monitoring, structured logs, and rate limiting.
- Add a production migration runner instead of running ad hoc setup commands.

License

This project was created as a FundsRoom full-stack case-study submission. Add a license before redistributing it outside the assignment or portfolio context.

Full-screen product screenshots

Sales overview



S

Stockflow

OPERATIONS PORTAL

Overview

Customers

Products

Stock movements

Challans

R

Riya Kapoor

Sales

→

Good morning, Riya

Products

Search name, SKU, category

CATALOGUE CONTROL

4 products

Products & inventory

PRODUCT ACTION	SKU / CATEGORY	STOCK	STATUS
A4 Copier Paper - 75 GSM Main Warehouse Request stock	PAP-A4-75 Stationery	49 units	Healthy
Blue Ball Pen - Pack of 10 Main Warehouse Request stock	PEN-BLU-10 Writing	34 units	Healthy
Hardbound Notebook - A5 Shelf B-12 Request stock	NOTE-A5-HB Stationery	43 units	Healthy
Permanent Marker - Black Shelf C-04 Request stock	MARK-BLK-01 Writing	6 units	Low Stock

Riya Kapoor

Sales

→

Warehouse overview

S

Stockflow

OPERATIONS PORTAL

Overview

Products

Stock movements

Challans

K

Kabir Shah

Warehouse

→

Good morning, Kabir

Operations at a glance

Pending requests

0

from Sales

Low-stock items

1

need attention

Warehouse tasks

1

in this queue

INVENTORY CONTROL

View products >

Low-stock items

PRODUCT	SKU	STOCK	STATUS
Permanent Marker - Black	MARK-BLK-01	6 units	Low Stock

SALES HANDOFFS

Pending stock requests

No pending requests.

Open request queue

WAREHOUSE NOTES

Keep stock movements traceable

Every approval, receipt, and adjustment is recorded with a reason in Stock movements.

View stock movements

Kabir Shah

Warehouse

→